
A BLOCK-SIZE CURRICULUM FOR DIFFUSION LANGUAGE MODEL PRETRAINING

Chang-Han Chen

University of California, Berkeley

Berkeley, CA 94720, USA

changhanc@berkeley.edu

ABSTRACT

Diffusion language models (dLMs) are pretraining-expensive. Common practice is either to train them from scratch at the target diffusion objective, or to adapt a pretrained autoregressive (AR) base with a brief diffusion finetune. We propose a simple two-stage block-size curriculum: train autoregressively (block size 1) for an early fraction p_{AR} of the total budget, then switch to small-block BD3 for the rest. Sweeping this fraction at 70M MoE scale, we find a robust optimum near $p_{\text{AR}}=0.30$ that beats both extremes across post-switch block sizes in the standard one-epoch regime. It is surprising that the optimum is independent of block length. To explain this, we hypothesize that the AR skills and representations transferable to diffusion are themselves independent of the BD3 block size. One might worry that AR initialization compromises diffusion’s performance in the data-constrained setting. Indeed, the naive recipe appears to fail: the AR phase overfits, the MoE router collapses at the handoff, and the BD3 continuation never catches scratch BD3. We show this is a tuning issue, not a fundamental one: tuning weight decay properly restores the curriculum to at least matching from-scratch BD3. We even suspect that modern data-efficiency techniques might improve the advantage of our curriculum in the multi-epoch setting.

1 INTRODUCTION

Diffusion language models (dLMs) are claimed to be “super data learners” (Ni et al., 2025; Gao et al., 2025): when unique training data is limited, they continue improving past the point where AR overfits, and small dLMs can eventually match or surpass AR baselines on downstream tasks. The catch is compute: dLMs are substantially more expensive to pretrain than AR models per training token. Masked-diffusion (MDLM) and block-diffusion (BD3LM) variants must learn any-order denoising, which is a strictly harder objective than left-to-right next-token prediction.

A natural compromise is a curriculum: spend an early fraction p_{AR} of the compute budget on AR training, then switch to a diffusion objective. This is essentially the recipe used by every recent at-scale dLM, but with the fraction implicitly fixed near 1:

- **BD3LM** (Arriola et al., 2025): 850K MDLM pretraining steps + 150K block-diffusion finetuning steps at block sizes $\{4, 8, 16\}$, i.e. $p_{\text{MDLM}}=0.85$ at sequence length 1024.
- **Dream-7B** (Ye et al., 2025): initialized from Qwen2.5-7B ($\sim 18\text{T}$ pretraining tokens) plus 580B continued-pretraining tokens for diffusion: effective $p_{\text{AR}} \approx 0.97$.
- **DiffuLLaMA / Efficient-DLM** (Fu et al., 2025): $\sim 65\text{B}$ adaptation tokens on a LLaMA-2 base ($\sim 2\text{T}$): $p_{\text{AR}} \approx 0.97$.
- **SDAR** (Cheng et al., 2025): 50B adaptation tokens on a Qwen3 base ($\sim 36\text{T}$): $p_{\text{AR}} \approx 0.999$.
- **LLaDA-2** (Bie et al., 2025): 3-phase block-WSD adaptation from a pretrained AR base, again very high p_{AR} .

We argue this convention is the wrong default *when training from scratch*. The implicit hypothesis is “train autoregressively as long as possible, then briefly adapt” — justified by the observation

that small-block BD3 ($b \leq 16$) is itself nearly autoregressive, plus the practical fact that an AR base usually already exists. But it leaves the middle of the p_{AR} range completely unexplored.

We treat p_{AR} as a tunable knob and sweep it at 70M MoE scale on ClimbMix, jointly with post-switch BD3 block lengths $b \in \{4, 16, 64, 256\}$. Our framing is a *block-size curriculum*: AR is the limit of block size 1, MDLM the limit of block size L , and BD3 with block length b is the in-between target. The curriculum trains at block size 1 for the first p_{AR} of the budget and at block size b for the rest. Our findings:

1. In the standard **one-epoch regime**, the validation-loss-optimal switch point is $p_{\text{AR}}=0.30$. It beats from-scratch BD3 across post-switch block lengths $b \in \{4, 16, 64\}$, beats AR-heavy curricula ($p_{\text{AR}} \geq 0.5$) at every post-switch block length, and is approximately invariant in the post-switch block length itself — consistent with the simple hypothesis that the AR skills and representations transferable to diffusion are themselves block-size-independent. Only at $b=256$ (half of sequence length) does the curriculum break down and scratch BD3 win.
2. In the **data-constrained regime** that motivates dLMs in the first place, the same recipe initially appears to fail: at unique-data pool sizes $U \in \{25\text{M}, 50\text{M}\}$ trained for 32 epochs each, naive $p_{\text{AR}}=0.30$ loses to scratch BD3 by 0.16–0.28 nats. Inspection shows the failure mode is AR-phase overfitting accompanied by MoE router collapse at the handoff, both of which are inherited by the BD3 continuation. Applying tuning weight decay properly (Muon WD on matrix parameters and a small Adam WD, with no router WD) restores the curriculum to within 0.01–0.02 nats of scratch.
3. We do not yet have data-constrained results at $b \in \{16, 64\}$. Based on the consistent direction of effects in the one-epoch regime — where curriculum’s margin over scratch grows with block size up to $b=64$ — we expect that with the AR-phase tuning fix, the data-constrained curriculum recovers a strict win at the larger block sizes too.

Our prescription is therefore: *when pretraining a block-diffusion model from scratch, start with 30% AR (with AR-phase weight decay if the training data is multi-epoch), then small-block BD3*. This is a strict improvement over current A2D practice, which uses an AR fraction at least $2.5\times$ larger than ours.

Caveats up front. (i) Our budget is fixed in optimizer steps, not wall-clock seconds. AR forward/backward passes are roughly $5\times$ cheaper than BD3 at our scale in our implementation, so a wall-clock-fixed comparison would weight AR even more favorably and the wall-clock-optimal p_{AR} would be larger than 0.30. The exact value depends on local hardware and BD3 implementation. (ii) The curriculum optimum almost certainly depends on absolute compute budget, model scale, and architecture choices we do not vary here.

2 RELATED WORK

Diffusion language models. MDLM (Sahoo et al., 2024; Shi et al., 2024) and BD3LM (Arriola et al., 2025) are the two dLM families we use. BD3LM partitions a sequence into blocks of length b , runs left-to-right block decoding, and applies discrete denoising within each block. The block size b interpolates between MDLM ($b=L$, full-sequence diffusion) and AR ($b=1$). Our curriculum is a temporal interpolation along the same axis.

A2D conversion. Dream-7B (Ye et al., 2025), DiffuLLaMA / Efficient-DLM (Fu et al., 2025), SDAR (Cheng et al., 2025), the next-block adaptation framework of Tian et al. (2025), and LLaDA-2 (Bie et al., 2025) all start from a pretrained AR base and add a comparatively brief diffusion-adaptation phase. We instead study A2D as a from-scratch curriculum, which strictly contains existing A2D as the high- p_{AR} corner.

Super-data-learner regime. Ni et al. (2025) and Gao et al. (2025) document that dLMs continue improving past the point where AR overfits in the multi-epoch data-constrained regime. Our data-constrained section operates in this regime to test whether the from-scratch curriculum recipe survives there.

3 METHOD

3.1 BLOCK-SIZE CURRICULUM

A two-stage block-size curriculum is parameterized by (p_{AR}, b) where $p_{\text{AR}} \in [0, 1]$ is the AR-phase fraction and b is the post-switch BD3 block length. With total budget T optimizer steps, the first $\lfloor p_{\text{AR}} T \rfloor$ steps are pure AR (block size 1, full causal attention, no mask tokens), and the remaining steps are BD3 with block length b . We also study the MDLM analogue, where the source phase is full-sequence MDLM (block size L) instead of AR. Pure-scratch BD3 ($p_{\text{AR}}=0$) is the natural baseline at each b .

The post-switch optimizer state is reset for the new objective in two senses: the loss function changes, and the LR multiplier changes. Short probe sweeps prefer `lr_mult=2.0` for AR→BD3 and scratch BD3, and `lr_mult=0.667` for MDLM→BD3 (Appendix A). Otherwise the optimizer continues as if uninterrupted.

3.2 ARCHITECTURE, DATA, OPTIMIZER

Model. A 70M-parameter MoE transformer with $(d_{\text{model}}, d_{\text{ff}}, n_{\text{layers}}, n_{\text{heads}}) = (768, 2048, 8, 12)$, RoPE positional embeddings, QK-norm, ReGLU MLPs, RMSNorm, value residuals, per-head attention gating, and depth-scaled layernorm. The MoE config uses a non-split router with z -loss weight 0.01. Attention uses cuDNN dense BD3 masks. Weight tying is on.

Data. ClimbMix BPE tokenizer (vocab 8192, mask token 8192). All runs use sequence length $L=512$ and effective batch size 512 (262,144 tokens per optimizer step). The one-epoch sweep uses the full 24-shard split (1.54B unique training tokens, ~ 1.7 epochs over $T=10,100$ steps). The data-constrained sweep uses two pool sizes drawn from a single shard split: $U=25\text{M}$ (first half of `shard_000000`; 32.2M actual training tokens) and $U=50\text{M}$ (all of `shard_000000`; $\sim 64.4\text{M}$ actual training tokens). Each is trained for 32 data epochs.

Optimizer. NorMuon+AdamW with grouped LR families. AdamW handles token embeddings, the tied LM head, value-embedding tables, and scalar parameters (RMSNorm gains, QK gains, value-residual scalars); NorMuon handles all matrix parameters. Adam betas (0.95, 0.99), eps 10^{-8} , Muon cautious WD 10^{-4} as a baseline default, momentum warmup $0.85 \rightarrow 0.95$. Default LR center (table, scalar, Muon) = (0.01, 0.005, 0.04), scaled by a single `lr_mult`. Schedule is 100-step linear warmup followed by constant LR.

One-epoch sweep grid. $p_{\text{AR}} \in \{0.30, 0.50, 0.80, 0.90, 0.95\}$ at $b \in \{4, 256\}$ for AR-init (full sweep at the two endpoint block sizes), plus $p_{\text{AR}}=0.30$ at $b \in \{16, 64\}$. The MDLM-init grid is filled in fully (5 p_{AR} values $\times$ 4 block sizes). Scratch BD3 baselines at $b \in \{4, 64, 256\}$.

Data-constrained sweep grid. 2 methods (scratch BD3 $b=4$, $p_{\text{AR}}=0.30$ $b=4$) $\times$ 2 U values $\times$ 2 weight-decay configurations = 8 cells. WD configurations are “no WD” (everything 0) and a tuned “split-WD” (AR phase: Muon WD 0.1, Adam WD 0.001, router Adam WD 0; BD3 phase: all WD 0).

4 MAIN RESULTS: ONE-EPOCH REGIME

4.1 AR-INIT GRID

For every populated row, the best column is $b=4$. For every populated column except $b=256$, the best row is $p_{\text{AR}}=0.30$. At $b=256$, scratch wins and AR-init is monotonically worse in p_{AR} . The overall best run is `ar_p030_b4_lr2_full` at 2.7275 best eval, beating same-block scratch by 0.32%.

Why we did not run $p_{\text{AR}} > 0.30$ at $b=64$. As b grows the post-switch task becomes less AR-like, so the optimal p_{AR} can only shift downward (and at $b=256$ it has shifted all the way to $p_{\text{AR}}=0$). Given that $p_{\text{AR}}=0.30$ already wins at $b=4$, the only remaining question at $b=64$ was whether $p_{\text{AR}}=0.30$ still beats scratch; the higher- p_{AR} rows would, under this monotonicity, only be worse. The single $p_{\text{AR}}=0.30$ point at $b=64$ confirms it: 2.9179 vs scratch 2.9467.

Table 1: AR→BD3 grid, best validation loss across the full 10.1k-step run. We swept p_{AR} fully at $b=4$ and $b=256$ and ran only $p_{\text{AR}}=0.30$ at $b=64$ (justified below). Bold: best per column.

p_{AR}	$b=4$	$b=64$	$b=256$
0.00 (scratch)	2.7364	2.9467	2.9408
0.30	2.7275	2.9179	2.9629
0.50	2.7717	—	3.0124
0.80	2.8511	—	3.1512
0.90	2.8786	—	3.2469
0.95	2.9924	—	3.4385

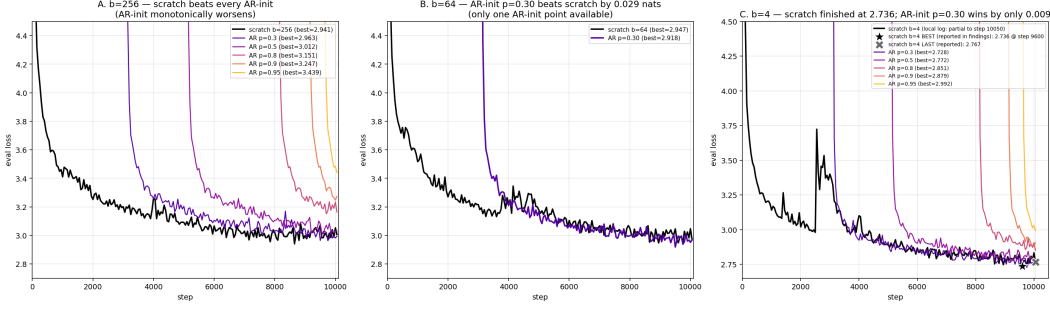


Figure 1: AR-init vs. scratch BD3 at three block lengths. **Left:** $b=256$, scratch beats every AR-init, monotonically in p_{AR} . **Middle:** $b=64$, AR-init at $p_{\text{AR}}=0.30$ beats scratch by 0.029 nats. **Right:** $b=4$, scratch finishes at 2.736; AR-init $p_{\text{AR}}=0.30$ wins by 0.009 nats.

Figure 1 shows the post-switch eval-loss curves at three block sizes. At $b=4$ and $b=64$, AR-init at $p_{\text{AR}}=0.30$ tracks or beats scratch BD3 from the post-switch step onward. At $b=256$, every AR-init run blows past scratch immediately after the switch and never recovers.

Figure 2 zooms into the late-training tails for the most directly comparable runs. At $b=4$, AR-init at $p_{\text{AR}}=0.30$ sits below scratch BD3 with the gap visible across the entire late-training window, and MDLM-init at $p_{\text{AR}}=0.30$ sits ~ 0.07 nats higher. At $b=256$, scratch lies clearly below MDLM-init at $p_{\text{AR}}=0.90$, consistent with the curriculum-breakdown story for this block size.

4.2 MDLM-INIT GRID

Table 2: MDLM→BD3 grid, best validation loss across the full 10.1k-step run. Row $p_{\text{MDLM}}=0$ is scratch BD3 at the matching block length (no MDLM phase). Bold: best per column.

p_{MDLM}	$b=4$	$b=16$	$b=64$	$b=256$
0.00 (scratch)	2.7364	—	2.9467	2.9408
0.30	2.8219	2.9497	2.9436	2.9528
0.50	2.8988	2.9610	2.9679	2.9630
0.80	2.9185	2.9645	2.9512	2.9358
0.90	2.9487	2.9811	2.9667	2.9505
0.95	3.0115	3.0292	2.9938	2.9570

Among MDLM-init runs ($p_{\text{MDLM}} > 0$), $p_{\text{MDLM}}=0.30$ wins at $b \in \{4, 16, 64\}$ and only $b=256$ prefers a later switch ($p_{\text{MDLM}}=0.80$). MDLM-init at $p_{\text{MDLM}}=0.30$ ties scratch at $b=64$ but loses by 1.6% at $b=4$ — AR-init transfers more cleanly to small-block BD3 than MDLM-init does. Linearly interpolating Table 2 places BD3LM’s published recipe (Arriola et al., 2025) ($p_{\text{MDLM}}=0.85$, $b \leq 16$) near 2.93 at $b=4$, i.e. $\sim 3.7\%$ worse than our $p_{\text{MDLM}}=0.30$ run.

Cross-track summary. Across Tables 1 and 2: (1) within every curriculum family the best post-switch block length is $b=4$, so the prescription is unambiguous on the BD3 side; (2) at every $b \leq 64$, $p_{\text{AR}}=0.30$ AR-init wins; (3) at $b=256$ the curriculum collapses and scratch wins.

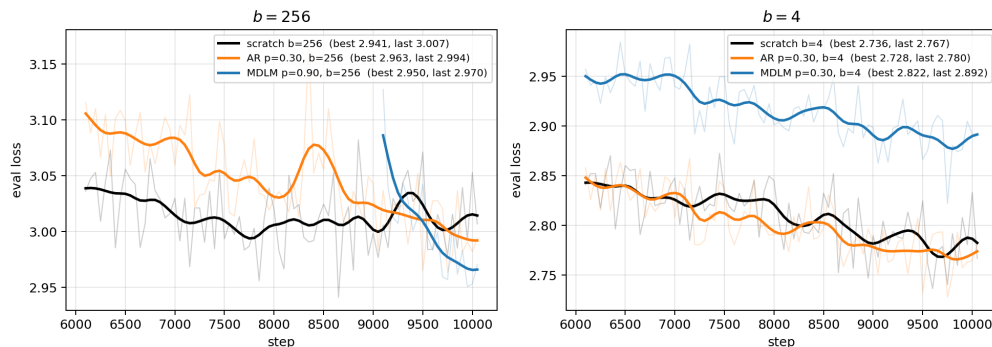


Figure 2: Late-training tails of the eval-loss curves over the last 4,000 optimizer steps. Curves are Gaussian-smoothed along the step axis with $\sigma \approx 106$ steps (FWHM ≈ 250 steps); the unsmoothed traces are shown faintly behind. **Left** ($b=256$): scratch BD3 beats MDLM-init at $p_{\text{MDLM}}=0.90$ across the late window. **Right** ($b=4$): AR-init at $p_{\text{AR}}=0.30$ tracks below scratch and clearly below MDLM-init at $p_{\text{MDLM}}=0.30$.

5 ANALYSIS

5.1 BLOCK-LENGTH INVARIANCE OF THE OPTIMUM

The most striking pattern in Tables 1 and 2 is that $p_{\text{AR}}=0.30$ wins at every block length $b \leq 64$, in both curriculum families. Naively this is surprising: at $b=4$ the post-switch task is nearly autoregressive, so a strong AR prior should be *very* valuable, suggesting a large p_{AR} . At $b=64$ the post-switch task is substantially less AR-like, so the AR prior should be *less* useful, suggesting a smaller p_{AR} . Empirically the optimum is the same.

A clean explanation: **the skills and representations AR learns that transfer to BD3 are themselves independent of the post-switch block length b** . The AR phase is identical regardless of b , so whatever it teaches that BD3 can use — vocabulary, syntax, semantics — is shared across all b , and saturates with strong diminishing returns. Past that early phase, AR’s marginal training refines mechanisms specific to its own objective, which need not transfer. To make this concrete, imagine training sequences of the form “ $A = p_1 \times p_2$ ” with $p_1=99,991$ and $p_2=100,003$ both prime, so $A=9,999,399,973$. To predict the right-hand side given “ $A =$ ”, AR must learn integer factorization, a hard mechanism. A BD3 model that sees “ $A = _ \times p_2$ ” only needs to divide. The two objectives reward fundamentally different computations, and AR’s late-stage factorization circuit is dead weight under BD3 regardless of b . Meanwhile, BD3 has its own slow-converging skills (mask handling, bidirectional within-block reasoning) that do not even make sense to AR. The optimal AR fraction is therefore set by AR-side dynamics — when the next AR step’s transferable contribution drops below what BD3 adaptation would buy at the same compute — and is, like AR’s transferable contribution itself, approximately invariant in b .

5.2 THE $b=256$ EXCEPTION

At half-sequence block ($b=256$), AR features fail to transfer at all: every AR-init run falls behind scratch immediately after the switch and never recovers (Fig. 3). At very large block size, the BD3 task is distributionally far from AR, so the question “when do AR’s marginal returns drop below BD3’s” answers itself: never, because AR doesn’t help here. MDLM-init partially recovers because MDLM features are themselves diffusion-flavored, so MDLM→BD3 retains some transferability even at large b .

6 DATA-CONSTRAINED SUBTLETY

The one-epoch sweep is in the regime where neither AR nor BD3 overfits. The motivating regime for dLMs is the opposite: small unique-data pools trained for many epochs, where AR is known

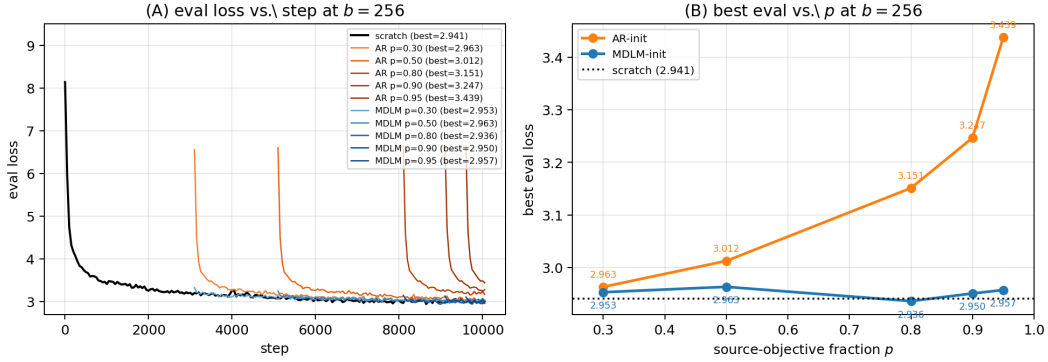


Figure 3: $b=256$. **(A)** Full eval-loss curves: AR-init runs blow past MDLM-init and scratch immediately after the switch and only the very early switches recover. **(B)** Best eval vs. source-objective fraction p : MDLM-init beats AR-init at every p , AR-init worsens monotonically with p , and only scratch ($p=0$) is competitive.

to overfit while diffusion continues to improve. We test the curriculum recipe in this regime at $U \in \{25\text{M}, 50\text{M}\}$, 32 epochs each, with $b=4$.

6.1 NAIVE $p_{\text{AR}}=0.30$ FAILS; AR-PHASE WEIGHT DECAY RESTORES IT

Table 3: Data-constrained runs at $b=4$, 32 epochs each. “Curriculum naive” is $p_{\text{AR}}=0.30$ AR $\rightarrow$ BD3 with no weight decay anywhere; “Curriculum tuned” uses split weight decay during the AR phase only (Muon WD 0.1, Adam WD 0.001, router Adam WD 0) and no WD during the BD3 phase. Each cell reports best eval @ epoch of best.

Method	$U=25\text{M}$	$U=50\text{M}$
Scratch BD3 $b=4$ (no WD)	3.2079 @ ep 29.3	2.9509 @ ep 26.1
Curriculum naive ($p_{\text{AR}}=0.30$, no WD)	3.4884 @ ep 17.9	3.1083 @ ep 26.7
Curriculum tuned ($p_{\text{AR}}=0.30$, AR-WD)	3.2231 @ ep 17.1	2.9603 @ ep 26.7
gap of naive to scratch	+0.2805	+0.1574
gap of tuned to scratch	+0.0152	+0.0094

The naive recipe loses to scratch by 0.28 nats at $U=25\text{M}$ and 0.16 nats at $U=50\text{M}$ — substantially worse than the one-epoch result, where the curriculum-scratch gap is at most 0.01 nats. The mechanism is visible in the AR phase: at $U=25\text{M}$ the no-WD AR prefix reaches its best eval at step 500 (epoch 4) at 3.5281 and then overfits to 3.9262 by the handoff, with the MoE router collapsing in parallel ($U=50\text{M}$ no-WD AR final router entropy 0.747, dropped-token fraction 0.030); the BD3 continuation inherits the damaged router and never catches scratch. Adding AR-phase weight decay flattens the AR phase, preserves the router (final entropy 1.001, dropped fraction 0.0004 at $U=50\text{M}$), and the BD3 continuation closes the gap to scratch dramatically: from 0.28 to 0.015 nats at $U=25\text{M}$, and from 0.16 to 0.009 nats at $U=50\text{M}$.

Figure 4 shows the eval-loss trajectories at $U=25\text{M}$ for the three methods.

Router health, not AR loss, predicts the BD3 continuation. At $U=50\text{M}$ the tuned AR phase has *worse* AR validation loss than the no-WD phase (3.290 vs 3.246 best), but the BD3 continuation from it is 0.148 nats *better*. AR validation loss is anti-correlated with what we actually care about. Since we have only run data-constrained curriculum at $b=4$, where the one-epoch margin was already small ($\sim 0.32\%$), the tuned curriculum lands close to scratch but does not strictly beat it; if the AR-phase WD fix continues to neutralize router collapse at $b=64$ (where the one-epoch margin is $\sim 1.0\%$), the larger margin should reappear.

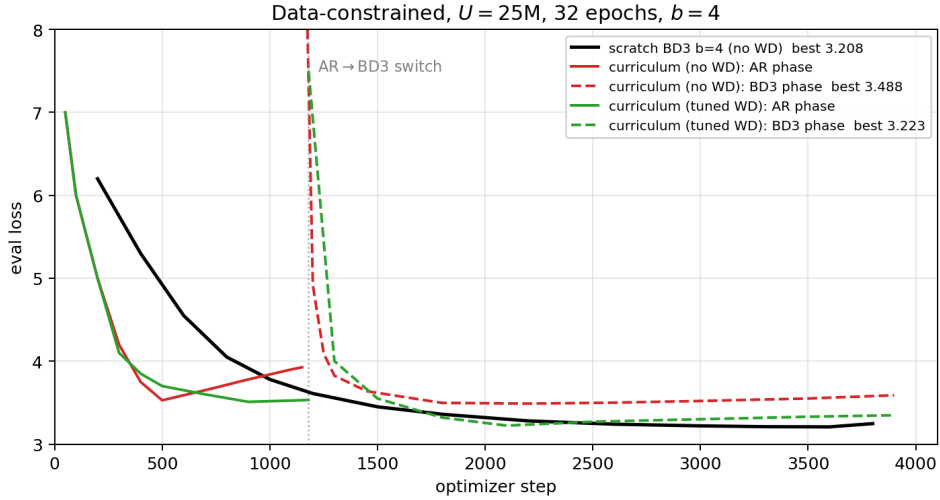


Figure 4: Data-constrained training curves at $U=25\text{M}$, 32 epochs, $b=4$. Black: scratch BD3, no WD. Red: naive curriculum at $p_{\text{AR}}=0.30$ with no WD; the AR phase peaks early then overfits, and the BD3 continuation never catches scratch. Green: tuned curriculum, same $p_{\text{AR}}=0.30$ but with AR-phase split-WD; the AR phase stays flat and the BD3 continuation tracks scratch closely. Switch occurs at step ≈ 1180 (30% of 3,930).

7 LIMITATIONS

Step-fixed, not wall-clock-fixed. Budget is in optimizer steps; AR passes are roughly $5\times$ cheaper than BD3 at our shape, so a wall-clock-fixed comparison would weight AR more favorably and the wall-clock-optimal p_{AR} would be larger than 0.30. The exact factor depends on local hardware and BD3 implementation; the qualitative direction is robust.

Single scale. The hypothesis has not been tested at scales beyond 70M. Vocabulary size 8,192 and sequence length 512 are intentionally small to maximize MFU and iteration speed at this size; whether $p_{\text{AR}}^*=0.30$ holds at larger model sizes, longer sequences, or larger vocabularies is open. As a downward-scale data point, we ran an analogous sweep at 1M parameters on TinyShakespeare and observed the same $p_{\text{AR}}=0.30$ optimum (Appendix B).

No downstream metric. At our scale, downstream performance on benchmarks such as HellaSwag is essentially noise; the prescription is therefore a validation-loss prescription.

MoE stability. Mixture-of-Experts is more sensitive to hyperparameters (LR, weight decay, router behavior) than a comparable dense transformer, and despite our best effort to tune, some of the under-performing configurations in this paper may simply reflect insufficient hyperparameter search rather than a fundamental disadvantage. We expect the qualitative findings to hold under additional tuning, but absolute numbers and cross-method gaps could shift.

More data-efficiency techniques. We show that AR-phase weight decay is sufficient to make the curriculum at least match scratch BD3 in the data-constrained regime. Recent work documents stronger data-efficiency interventions that are orthogonal to ours: Kim et al. (2025) show that aggressive weight decay ($\sim 30\times$ standard practice), parameter scaling, and ensembling-with-distillation extract substantially more performance from a fixed data pool, and the open NanoGPT-Slowrun benchmark (Q Labs, 2025) aggregates further community-found tricks for the same regime. These should stack on top of our curriculum and would tighten the data-constrained results we report. As a side remark: based on the data-constrained results in this paper plus the data-efficiency techniques surveyed in Kim et al. (2025) and (Q Labs, 2025), we suspect that diffusion does not in fact strictly beat AR in this regime once both objectives are given the same regularization and ensembling treatment — a claim worth revisiting in the next iteration.

8 CONCLUSION

When pretraining a block-diffusion language model from scratch, the right default is a two-stage block-size curriculum with $p_{\text{AR}} \approx 0.30$: train autoregressively for the first 30% of the budget, then switch to small-block BD3. In the standard one-epoch regime this beats both extremes (pure-scratch BD3 and AR-heavy curricula) across post-switch block sizes, and the optimum itself is approximately invariant in the BD3 block length — a finding consistent with the AR skills and representations transferable to diffusion being themselves block-size-independent. In the multi-epoch data-constrained regime the recipe initially fails because the AR phase overfits and the MoE router collapses at the handoff; tuning weight decay properly restores it to at least matching scratch BD3, with reason to expect that the one-epoch curriculum margin reappears at larger block sizes once tuned. The recipe is much closer to optimal than current AR-heavy A2D practice, and is a single-knob prescription: $p_{\text{AR}}=0.30$, with AR-phase weight decay if data-constrained.

ACKNOWLEDGMENTS

Experiments were run on shared $4\times\text{H100}$ nodes rented on Vast.ai.

REFERENCES

- Marianne Arriola, Aaron Gokaslan, Subham Sekhar Sahoo, and Volodymyr Kuleshov. Block diffusion: Interpolating between autoregressive and diffusion language models. *arXiv preprint arXiv:2503.09573*, 2025.
- Tiwei Bie, Maosong Cao, Kun Chen, Lun Du, Mingliang Gong, Zhuochen Gong, et al. LLaDA 2.0: Scaling up diffusion language models to 100b. *arXiv preprint arXiv:2512.15745*, 2025.
- Shuang Cheng, Yihan Bian, Dawei Liu, Linfeng Zhang, and Qian Yao. SDAR: A synergistic diffusion-autoregression paradigm for scalable sequence generation. *arXiv preprint arXiv:2510.06303*, 2025.
- Yonggan Fu, Lexington Whalen, Zhifan Ye, Xin Dong, Shizhe Diao, and Song Han. Efficient-DLM: From autoregressive to diffusion language models, and beyond in speed. *arXiv preprint arXiv:2512.14067*, 2025.
- Zitian Gao, Haoming Luo, Lynx Chen, Jason Klein Liu, Ran Tao, Joey Zhou, and Bryan Dai. What makes diffusion language models super data learners? *arXiv preprint arXiv:2510.04071*, 2025.
- Konwoo Kim, Suhas Kotha, Percy Liang, and Tatsunori Hashimoto. Pre-training under infinite compute. *arXiv preprint arXiv:2509.14786*, 2025.
- Jinjie Ni, Qian Liu, Longxu Dou, Chao Du, Zili Wang, Hang Yan, Tianyu Pang, and Michael Qizhe Shieh. Diffusion language models are super data learners. *arXiv preprint arXiv:2511.03276*, 2025.
- Q Labs. NanoGPT Slowrun: Language modeling with limited data, infinite compute. <https://github.com/qlabs-eng/slowrun>, 2025.
- Subham Sekhar Sahoo, Marianne Arriola, Yair Schiff, Aaron Gokaslan, Edgar Marroquin, Justin T Chiu, Alexander Rush, and Volodymyr Kuleshov. Simple and effective masked diffusion language models. In *Advances in Neural Information Processing Systems*, 2024.
- Jiaxin Shi, Kehang Han, Zhe Wang, Arnaud Doucet, and Michalis K Titsias. Simplified and generalized masked diffusion for discrete data. In *Advances in Neural Information Processing Systems*, 2024.
- Yuchuan Tian, Yuchen Liang, Shuo Zhang, Yingte Shu, and Guangwen Yang. From next-token to next-block: A principled adaptation path for diffusion LLMs. *arXiv preprint arXiv:2512.06776*, 2025.
- Jiacheng Ye et al. DREAM: Diffusion rectification and estimation-adaptive models. *arXiv preprint arXiv:2410.16168*, 2025.

A LR-MULTIPLIER PROBES

We selected `lr_mult` by short probes prior to the main grid.

Table 4: LR-multiplier probes for MDLM→BD3 switches. Probe length 1500 steps, $p_{\text{MDLM}}=0.30$, $b=64$. `lr_mult`=0.6667 chosen.

<code>lr_mult</code>	Best eval	Notes
0.22	3.1716	Too slow / low, terminated
0.6667	3.0973	Selected
2.00	3.1318	Viable but worse
6.00	3.3373	Bad, terminated

The AR→BD3 switch and scratch BD3 baseline use `lr_mult`=2.0, which matches BD3LM’s scratch baseline LR. The asymmetry between AR-init and MDLM-init switch LRs is consistent with MDLM features sitting closer to BD3 in optimization geometry, so a smaller LR avoids disturbing them.

B SMALLER-SCALE CURRICULUM RESULT ON TINYSHAKESPEARE

We ran an analogous AR→block-MDLM curriculum sweep on TinyShakespeare at much smaller scale (0.3M and 1M-parameter transformers) under an IsoFLOP budget of $C = 6 \times 10^{14}$, varying total steps and the AR fraction p_{AR} . The same minimum at $p_{\text{AR}}=0.30$ appears (Table 5, Figure 5), supporting the hypothesis at this much smaller scale.

Table 5: TinyShakespeare validation cross-entropy for AR→block-MDLM curriculum. Bold: best per row.

Model size	1,200 total steps				
	$p_{\text{AR}}=0$	$p_{\text{AR}}=0.1$	$p_{\text{AR}}=0.3$	$p_{\text{AR}}=0.5$	$p_{\text{AR}}=0.7$
0.3M	2.045	1.973	1.957	1.966	2.006
1M	1.891	1.871	1.814	1.839	1.865

Model size	4,000 total steps			
	$p_{\text{AR}}=0$		$p_{\text{AR}}=0.3$	$p_{\text{AR}}=0.5$
0.3M	1.860	—	1.812	1.830
1M	1.733	—	1.690	1.692

Figure 3. Curriculum sweet spot stays at $p_{\text{ar}} = 0.3$

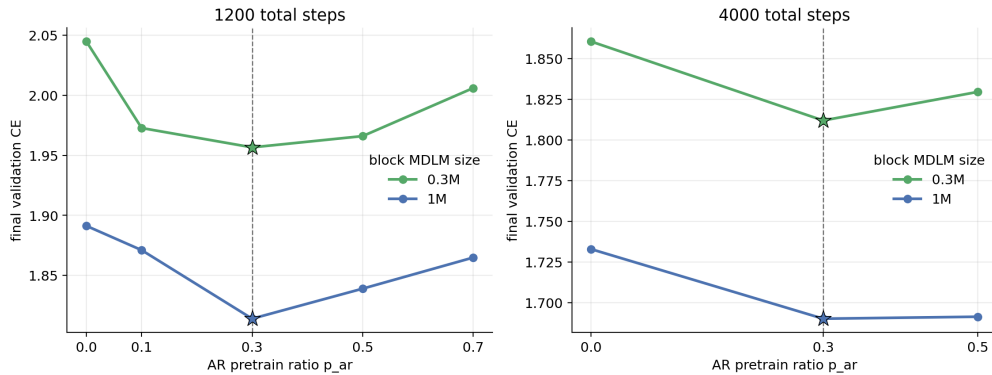


Figure 5: TinyShakespeare validation cross-entropy as a function of AR fraction p_{AR} , at two model sizes (0.3M and 1M parameters) and two total budgets (1,200 and 4,000 steps). The same $p_{\text{AR}}=0.30$ minimum appears at both sizes and both budgets.

The TinyShakespeare runs are character-level rather than BPE, the post-switch objective is full-block MDLM (block size 4 within sequence length 32) rather than the BD3 used in the main body, and the model and dataset are several orders of magnitude smaller. We include this only as a downward-scale data point, not as evidence of scale generalization upward.

C DATA-CONSTRAINED TUNING DETAILS

C.1 $U=25\text{M}$ AR-PHASE WEIGHT-DECAY SWEEP

We swept AR-phase weight-decay configurations at $U=25\text{M}$ with a 1,000-step AR probe before committing to the full curriculum. The split-WD configuration with Muon WD 0.1, Adam WD 0.001, router Adam WD 0 was the best AR-only result.

Table 6: AR-only probes at $U=25\text{M}$, 1,000 steps each. Selected configuration in bold.

Run name	Muon WD	Adam WD	Router WD	Best AR eval	Best step
...muw0p1_adamwd0_ar	0.1	0	0	3.5224	900
...muw0p1_adamwd0p01_routerwd0_ar	0.1	0.01	0	3.6374	550
... muw0p1_adamwd0p001_routerwd0_ar	0.1	0.001	0	3.4609	850
...muw0p05_adamwd0p001_routerwd0_ar	0.05	0.001	0	3.4673	850

Applying WD to all groups (including the router) was uniformly worse. Applying it to nothing (the naive setting) overfits at step ~ 500 and degrades thereafter. Applying it to Muon-updated matrices and a small amount to Adam-updated table parameters, with router WD off, was the best combination and what we use throughout the data-constrained sweep.

C.2 SCRATCH BD3 WITH ALL-WD-0.1 IS UNSTABLE

We initially considered applying WD to BD3 as well. At $U=25\text{M}$, scratch BD3 with all WD set to 0.1 reaches a worse minimum (3.3613 vs 3.2079) and trains unstably to a final eval of 4.4145. BD3 should use no WD in this setting. The mismatch between the AR phase (needs WD) and the BD3 phase (does not) is a real asymmetry; the curriculum benefits from per-phase weight-decay tuning, not a single global WD.

D ARCHITECTURE AND REPRODUCIBILITY

All curriculum runs in this paper share a fixed architecture and optimizer recipe. Table 7 lists the values.

D.1 ARCHITECTURE-INTERVENTION DELTAS (AR OBJECTIVE)

Each architectural feature in Table 7 contributes non-trivially to AR validation loss at a matched 5,100-step single-epoch reference setup (same shape, optimizer, and data as Table 7, but trained on the full 1.54B-token split rather than a small data pool). Table 8 reports best, final, and average-over-last-250-steps eval loss for each intervention plus the best-eval improvement over the plain transformer baseline. The combined recipe in the last row of Table 8 is what we use as the source architecture for every curriculum run in the main body.

QK-norm + value residuals + per-head gating + LN scaling alone account for roughly 3% of the improvement; MoE alone for $\sim 1.6\%$; replacing value residuals with a value-embedding table at the same attention-feature setting is roughly neutral on best eval. The combination is the strongest configuration we tested.

D.2 MULTI-STAGE BLOCK-SIZE RAMPS ARE NOT WORTH IT

A natural alternative to the two-stage curriculum is a multi-stage block-size ramp. We ran one such ramp (run ar_p050_multiblock_b4_b16_b64_b256_lr2_decay) at the same 70M MoE

Table 7: Model, data, and optimizer configuration. The same configuration is used for every run in the paper; only the curriculum knobs (p_{AR} , post-switch block length b , weight-decay group settings) vary.

Field	Value
Architecture	70M-parameter Mixture-of-Experts transformer
$d_{\text{model}}, d_{\text{ff}}, n_{\text{layers}}, n_{\text{heads}}$	768, 2048, 8, 12
Positional embedding	RoPE
Attention features	QK-normalization, value residuals, per-head gating, depth-scaled layernorm
MLP	ReGLU
Normalization	RMSNorm
Attention implementation	cuDNN dense (with BD3 block mask)
MoE: experts per layer	4, top-1 routing, capacity factor 1.25
MoE: layer placement	every other layer (4 MoE layers out of 8)
MoE: router input on BD3	shared between x_t and x_0 streams (not split)
MoE: auxiliary loss	router z -loss with weight 0.01
Weight tying	embedding $\leftrightarrow$ LM head
Tokenizer	byte-pair encoding, vocab 8,192, mask token 8,192
Data	ClimbMix
Sequence length	512 tokens
Effective batch size	512 sequences (262,144 tokens / optimizer step)
Optimizer (matrix parameters)	NorMuon, momentum warmup 0.85 $\rightarrow$ 0.95
Optimizer (embeddings + scalars)	AdamW, $(\beta_1, \beta_2, \epsilon) = (0.95, 0.99, 10^{-8})$
Base learning rate (embedding tables, scalars, matrices)	(0.01, 0.005, 0.04)
LR multiplier, AR phase	5.0
LR multiplier, scratch BD3 / AR $\rightarrow$ BD3 post-switch	2.0
LR multiplier, MDLM $\rightarrow$ BD3 post-switch	0.6667
Default Muon weight decay	10^{-4}
Schedule	100-step linear warmup, then constant
One-epoch budget	10,100 steps ($\sim 2.65\text{B}$ tokens; ~ 1.7 epochs over 1.54B unique)
Data-constrained budget	32 epochs per pool: $U=25\text{M} \Rightarrow 3,930$ steps; $U=50\text{M} \Rightarrow \sim 7,860$ steps
Hardware	4 $\times$ NVIDIA H100 (80GB)

Table 8: Architectural ablation under the AR objective. “Best” is the lowest eval loss observed during a 5,100-step training run; “avg-250” averages eval losses logged in the last 250 training steps; Δ best is the relative best-eval improvement over the plain transformer baseline (first row). The first two rows are sourced from W&B; the remaining rows are computed locally from training logs.

Configuration	Best	Final	Avg-250	Δ best
Plain transformer (RoPE, ReGLU, RMSNorm only)	2.8529	2.8841	—	—
+ QK-norm, value residuals, per-head gating, LN scaling	2.7686	2.8000	—	+2.96%
+ QK-norm, value embedding, per-head gating, LN scaling	2.8480	2.8895	2.8798	+0.17%
+ Mixture-of-Experts (no other extras)	2.8062	2.8380	2.8331	+1.64%
+ MoE and the four attention features (used in main body)	2.7523	2.7792	2.7823	+3.53%

setup as the main body: AR for the first 5,100 steps, then BD3 at $b=4$ (5,100–6,200), $b=16$ (6,200–7,200), $b=64$ (7,200–8,200), $b=256$ (8,200–10,100). Figure 6 shows the eval-loss trajectory. The best eval loss across the entire run is 2.9038 (during the $b=4$ stage at step 6,100), while the single-stage AR-init $b=4$ winner from the main body reaches 2.7275. The ramp’s eval loss never goes below the single-stage line at any step, and once the curriculum moves past $b=4$ the loss only gets worse. Multi-stage block-size ramps are a worse use of budget than a single AR $\rightarrow$ small-block transition, which is why we restrict the main 70M sweep to two-stage curricula.

Multi-block ramp ($p_{\text{AR}} = 0.50$, then $b = 4 \rightarrow 16 \rightarrow 64 \rightarrow 256$) stays strictly above the single-stage optimum

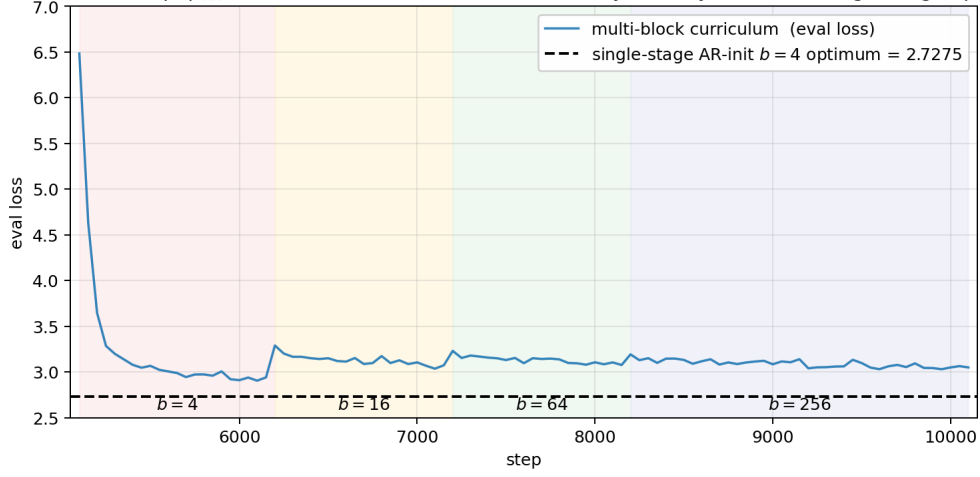


Figure 6: Eval-loss trajectory for the multi-block ramp (AR for 5,100 steps, then BD3 at $b=4 \rightarrow 16 \rightarrow 64 \rightarrow 256$, 1,000–1,900 steps per stage). Background colors mark each block-size stage. The dashed line is the best validation loss of the single-stage AR→BD3 $p_{\text{AR}}=0.30$, $b=4$ run (`ar_p030_b4_lr2_full` = 2.7275, Table 1). The multi-block run’s eval loss stays strictly above the single-stage line throughout.